Assignment 2 – Git & GitHub Fundamentals

Group Report: Unit Converter

Field	Details
Group Size	3 members (3-member path)
Chosen Theme	Unit Converter
Full Names	El Mehdi Rahmoune; Hanae Cherkaoui; Hiba El Adraoui
Student IDs	El Mehdi Rahmoune: 156875; Hanae Cherkaoui: 162125; Hiba El Adraoui: 168046
GitHub Usernames	mehdibuilds; HanaeCherk90; HibaEladraoui
GitHub Repository URL	https://github.com/mehdibuilds/unit-converter
GitHub Pages URL	https://mehdibuilds.github.io/unit-converter/

1. Project & Role Distribution

We built a small command-line **unit converter** in Python, tested with pytest, checked by GitHub Actions and described on a GitHub Pages site. The group followed the **3-member path**: three comparable feature areas, one per member, each with its own module and test file so the features could be developed and merged independently.

Member	Feature	Files
El Mehdi Rahmoune (A)	Length conversion (mm, cm, m, km, in, ft, yd, mi)	`converters/length.py`, `tests/test_length.py`
Hanae Cherkaoui (B)	Temperature conversion (Celsius, Fahrenheit, Kelvin, absolute-zero check)	`converters/temperature.py`, `tests/test_temperature.py`
Hiba El Adraoui (C)	Weight conversion (mg, g, kg, t, oz, lb, st, negative-weight check)	`converters/weight.py`, `tests/test_weight.py`

Shared files (README, `.gitignore`, `requirements.txt`, `pytest.ini`, `app.py`, `converters/__init__.py`) were created by Mehdi in the initial commit on `main`. Extra roles: Mehdi created the repository and ruleset; Hanae built the CI workflow; Hiba built the GitHub Pages page.

2. Branches, Commits & Contributions

Member	Feature branch and other branches	Meaningful commits
El Mehdi Rahmoune	`feature/length`; `docs/report`	“Add length conversion with unit validation”; “Add unit tests for length conversion”; initial commit “Add initial project structure and README” on `main`; “Add group report PDF” (report PR)

Member	Feature branch and other branches	Meaningful commits
Hanae Cherkaoui	`feature/temperature`; `ci/add-workflow`; `status/core-done`; `docs/final-update`	“Add temperature conversion with absolute-zero check”; “Add unit tests for temperature conversion”; “Add GitHub Actions workflow to run pytest”; “Fix CI: install requirements before running pytest”; “Update project status: core converters done”; “Update project page status to v1.0”
Hiba El Adraoui	`feature/weight`; `status/ci-in-progress`; `docs/project-page`	“Add weight conversion”; “Add unit tests for weight conversion”; “Reject negative weights with ValueError”; “Add test for negative weight”; “Update project status: tests and CI in progress”; “Resolve README status conflict with origin/main”; “Add GitHub Pages project page”

Every member made at least two commits on their own feature branch, started from the latest `main`, and no feature was developed directly on `main`.

3. Pull Requests & Code Review

Pull Request	Author	Reviewer	Outcome
#1 Add length conversion feature (Feature A)	Mehdi	Hanae	Line comment, approved, merged
Add temperature conversion feature (Feature B)	Hanae	Hiba	Line comment, approved, merged
Add weight conversion feature (Feature C)	Hiba	Hanae	**Request changes**, fixed on same branch, approved, merged
Add CI workflow	Hanae	Hiba	Line comment, approved, merged
Update project status (core done)	Hanae	Hiba	Approved, merged first
Update project status (tests and CI)	Hiba	Mehdi	Conflict resolved, line comment, approved, merged
Add GitHub Pages project page	Hiba	Hanae	Approved, merged
Update project page status to v1.0	Hanae	Hiba	Approved, merged; live site updated

Request-changes cycle

Hanae reviewed Hiba's weight PR in the Files changed tab and left a line comment on `convert()` asking that negative weights raise `ValueError` and that a test cover it, then submitted **Request changes**. Hiba did not open a new PR: she committed “Reject negative weights with ValueError” and “Add test for negative

weight" on the **same branch** `feature/weight` and pushed. The existing PR updated itself, Hanae re-reviewed the new commits and approved, and the PR was merged.

4. Merge Conflict

`README.md` contains the line **Project status: Development in progress.** Hanae and Hiba each created a branch from the same version of `main` and changed that exact line differently:

- Hanae (`status/core-done`): "Project status: Core converters implemented."
- Hiba (`status/ci-in-progress`): "Project status: Tests and CI in progress."

Hanae's PR was approved and merged first. Hiba then ran `git fetch origin` and `git merge origin/main` on her branch, and Git reported a conflict in `README.md`. **Current** (HEAD) was Hiba's line from her own branch; **Incoming** (origin/main) was Hanae's line already merged into `main`. Both statements were true, so the group combined them into one line:

```
Project status: Core converters implemented; tests and CI in progress.
```

Hiba staged the file, committed "Resolve README status conflict with origin/main", pushed, and Mehdi (the third member) reviewed and approved the PR before it was merged.

5. Unit Tests

The suite has **18 pytest tests** (6 per feature) in `tests/test_length.py`, `tests/test_temperature.py` and `tests/test_weight.py`. All pass locally and in CI.

Kind	Examples
Normal behavior	1.5 km → 1500 m; 1 mi → 1.609344 km; 100 °C → 212 °F; 32 °F → 273.15 K; 1 kg → 2.20462 lb; 100 g → 3.5274 oz
Edge / boundary	Exactly absolute zero is allowed (0 K = −273.15 °C); zero length and weight convert to 0; converting a unit to itself; unit names are case-insensitive
Invalid input / errors	Unknown unit → `ValueError`; non-numeric value → `TypeError`; boolean value → `TypeError`; temperature below absolute zero → `ValueError`; NaN → `ValueError`; negative weight → `ValueError`

6. Continuous Integration

The workflow `.github/workflows/tests.yml` runs on every **push** and every **pull request**. It has one job named `pytest` on an **ubuntu-latest** runner with these steps: check out the repository, set up Python 3.12, install dependencies (`pip install -r requirements.txt`), and run `pytest -v`.

Failed run: the first version of the workflow (Hanae's first commit on `ci/add-workflow`) did not contain the install step, so the runner had no pytest and the "Run the test suite" step failed because pytest was not installed. **Fix:** Hanae added the "Install dependencies" step, committed "Fix CI: install requirements before running pytest" and pushed to the same PR; the run then passed.

7. Ruleset & Protected main

The active branch ruleset **"Protect main"** targets the default branch with an empty bypass list. Its final rules are:

- Restrict deletions
- Block force pushes
- Require a pull request before merging, with **1 required approval**
- Require status checks to pass: **pytest** (added after the workflow had run successfully)

A PR could therefore not be merged until another member approved it **and** the `pytest` check had passed. Direct pushes to `main` were rejected by GitHub (this was seen when committing to `main` after the ruleset was created).

8. Continuous Deployment

GitHub Pages is configured under Settings → Pages as **Deploy from a branch**, branch `main`, folder `/docs`. Every merge to `main` that changes `docs/` triggers GitHub's "pages build and deployment" workflow, which publishes the site at https://mehdibuilds.github.io/unit-converter/.

For the final verification, Hanae created `docs/final-update` and changed the status badge on `docs/index.html` from "Status: development in progress" to "Status: v1.0 - all features complete". After Hiba approved and the check passed, the PR was merged; once GitHub's Pages deployment finished, the live page showed the new status without any manual step.

9. Contribution Summary

	El Mehdi Rahmoune	Hanae Cherkaoui	Hiba El Adraoui
Feature(s)	Length; repo, ruleset, Pages setup, report	Temperature; CI workflow	Weight; merge-conflict resolution; Pages page
Branch(es)	`feature/length`, `docs/report`	`feature/temperature`, `ci/add-workflow`, `status/core-done`, `docs/final-update`	`feature/weight`, `status/ci-in-progress`, `docs/project-page`
PRs authored	#1 length conversion; report PR (docs/report)	Temperature feature; CI workflow; status (core done); final page update	Weight feature; status (tests and CI); GitHub Pages page
PRs reviewed	Conflict-resolution PR (status: tests and CI)	#1 length; weight (Request changes); Pages page	Temperature; CI; status (core done); final update
Test contribution	6 tests (length)	6 tests (temperature)	6 tests (weight, incl. negative-weight test added after review)

10. Fair Use of AI

The group used an AI assistant (Claude) to draft the starter code for the three converter modules, the pytest tests, the GitHub Actions workflow and the step-by-step instruction sheets, and to draft the text of

this report. The prompts asked for a simple unit-converter project split into three comparable features, with tests covering normal, edge and invalid cases, and a CI workflow. We kept the structure and code as generated, but changed the workflow so that the first version deliberately omitted the install step, and left negative-weight validation out of the first weight version so that it could be requested in review. We verified the result by running all 18 pytest tests locally, by the GitHub Actions runs, and by reading and commenting on each other's code in pull requests. Each member is responsible for understanding the code, tests, Git commands, workflow and review actions attributed to them.

11. Sources & Reflection

- GitHub Docs, About rulesets: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-rulesets/about-rulesets
- GitHub Docs, GitHub Actions: https://docs.github.com/en/actions
- GitHub Docs, GitHub Pages: https://docs.github.com/en/pages
- pytest documentation: https://docs.pytest.org

The hardest part was coordinating the merge conflict: both branches had to start from the same version of `main`, the first PR had to be merged before the second was resolved, and the required `pytest` check and approval had to pass each time. If a pull request, merge, test or workflow behaves unexpectedly, we would first check the PR's merge box (missing approval or failing check), then whether the branch is up to date with `main`, and then the Actions log of the failing step.